

# Objectives

- Working with Functions
- Creating and Using Modules
- Managing Packages with npm

## Functions in Node.js

### Introduction

A function is a reusable block of code that performs a specific task. Functions help avoid repetition and improve code readability, structure, and maintainability.

### Creating Functions

We create functions in JavaScript using the `function` keyword. There are two ways to do this, we can use a **function declaration** or a **function expression** (where we store the result in a variable).

### Function Declaration

We do a **function declaration** by using the keyword `function`, followed by the name of the function, then parentheses `()`. Inside them, we add the **parameters** that the function will need. After that, we use curly braces `{}` to enclose the code that the function will run.

```
function greet(name) {  
    return "Hello, " + name;  
}  
greet("Alice"); // calling the function
```

### Function Expression

A function expression is another way to declare a function. First, we create the function, then assign it to a variable. Later, we use that variable to call the function.

```
const greet = function(name) {  
    return "Hello, " + name;  
}
```

```
};  
greet("Alice"); // calling the function
```

## Arrow Function

An arrow function provides a shorter syntax for writing functions. We create it by assigning the function to a variable, using parentheses `()` for parameters (if needed), followed by the `=>` arrow and the function body. For single expressions, curly braces `{}` and the `return` keyword can be omitted.

```
const greet = name => `Hello, ${name}`;  
greet("Alice");
```

## Scope :

In programming, **scope** refers to the region of a program where a particular variable or function can be accessed and used.

### Global scope

Global scope represents the top-level scope of a script file. Any variable or constant created outside a function belongs to this scope. These are referred to as **global variables**.

Global variables can be accessed and used by any function or block of code within the script.

### Local scope

Local scope means a variable can only be seen and used inside the function where it is created, if a variable is made inside a function, it's called a **local variable**.

```
const globalMessage = "This is global";  
  
function printMessage() {  
  console.log(globalMessage); // Accessible  
}  
  
function printSum(a, b) {  
  const result = a + b;        // Local  
  console.log(result);  
}  
  
printSum(3, 5);
```

```
console.log(result); // we will get error: not defined
```

## Parameters

Parameters are the names listed in a function's definition. We use them to pass data into a function.

```
function greet(name, age) {  
    return `Hello, ${name}! You are ${age} years old.`;  
}
```

The parameters here are `name` and `age`.

We pass data to our functions using arguments. These arguments represent the actual values we provide when we call a function. When we call a function with fewer arguments than we declared parameters, the missing parameters in our function become `undefined`.

## Default Parameters

In JavaScript, we can set default values for our function parameters. When we declare a function, we assign these defaults directly in the parameter list. If we call the function without providing an argument (or pass `undefined`), the default value is used instead.

```
function greet(name = "Guest", greeting = "Hello") {  
    console.log(`${greeting}, ${name}!`);  
}  
  
greet(); // "Hello, Guest!" (uses both defaults)  
greet("Alice"); // "Hello, Alice!" (uses default greeting)
```

## The Arguments Object

JavaScript functions have access to a special object called `arguments`, which contains all arguments passed to the function, regardless of how many parameters were declared:

```
function logAllArguments() {  
    for (let i = 0; i < arguments.length; i++) {  
        console.log(`Argument ${i}: ${arguments[i]}`);  
    }  
}
```

```
logAllArguments("apple", "banana", "cherry");  
// Outputs:  
// Argument 0: apple  
// Argument 1: banana  
// Argument 2: cherry
```

The `arguments` object is array-like but not an actual array. If we want use array methods with it, we first need to convert it to a real array.

```
function sumAll() {  
    const args = Array.from(arguments);  
    console.log(args)  
}
```

However, the `arguments` object has some limitations:

- It's not available in arrow functions
- It can be confusing in nested functions
- It doesn't have array methods by default

## Arbitrary Number of Arguments

If we want a function to take an arbitrary number of arguments we can use the rest parameter, this parameter is created by three dots ( `...` ) followed by a parameter name, it will create a real array containing all remaining arguments

```
function greet(greeting, ...names) {  
    console.log(`${greeting}, ${names.join(' and ')}!`);  
}  
greet('Hello', 'Alice', 'Bob'); // "Hello, Alice and Bob!"
```

## Argument Destructuring

We can extract specific properties from objects directly in a function's parameters using **destructuring assignment**. This technique allows us to unpack values from object properties into distinct variables by specifying the exact keys we need within curly braces `{}`.

```
function printUser({ name, age, country = "Unknown" }) {  
    console.log(`${name} (${age}) from ${country}`);  
}
```

```
const user = { name: "Alice", age: 28 };
printUser(user); // "Alice (28) from Unknown"
```

## Argument Passing: By Value vs. By Reference

In JavaScript, primitive values (strings, numbers, booleans) are passed *by value*. This means when we pass them to a function, we're working with a copy of the original value, modifications inside the function won't affect the original variable.

```
function modifyPrimitive(num) {
    num = num * 2;
    console.log(num); // 10
}
let x = 5;
modifyPrimitive(x);
console.log(x); // Still 5, unchanged
function modifyObject(obj) {
    obj.name = "Modified";
    console.log(obj.name); // "Modified"
}
let user = { name: "Original" };
modifyObject(user);
console.log(user.name); // "Modified", the original object was changed
```

## Functions Returning Values

### Return Statements in Functions

Every JavaScript function returns a value. If a return statement isn't explicitly provided, the function returns `undefined` by default

```
function greet(name) {
    console.log(`Hello, ${name}!`);
}
const result = greet("Alice"); // Logs: "Hello, Alice!"
console.log(result); // undefined
```

The `return` statement immediately exits the function any statement after it will not run

```
function add(a, b) {  
    return a + b; // Code after return is never executed  
    console.log("This will never run");  
}  
  
const sum = add(5, 3); // 8
```

## Returning Multiple Values

We can return multiple value from function by returning them as array or object

```
function getUserStats(user) {  
    return {  
        name: user.name,  
        postsCount: user.posts.length,  
        followersCount: user.followers.length  
    };  
}
```

## Returning Function

In JavaScript, we can return functions from other functions just like any other value. This enables powerful patterns like function factories, closures, and higher-order functions.

Function factories

Function factory is just a function that returns another function a "factory" for making functions!

```
function multiplier(factor) {  
    return function(x) {  
        return x * factor;  
    };  
}  
  
const double = multiplier(2);  
const triple = multiplier(3);  
  
console.log(double(5)); // 10  
console.log(triple(5)); // 15
```

## Closure

Closures allow a function to "remember" and access variables from its lexical scope even when

the function is executed outside of that scop

```
function counter() {  
  let count = 0;  
  return function() {  
    count += 1;  
    return count;  
  };  
}  
  
const increment = counter();  
console.log(increment()); // 1  
console.log(increment()); // 2
```

## Higher-order functions

A higher-order function is a function that does at least one of the following:

1. Takes another function as an argument
2. Returns a function

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
function processUserInput(callback) {  
  const name = "Alice";  
  console.log(callback(name));  
}  
  
processUserInput(greet);
```

## Recursive Functions

Recursive function are special function that have ability to call theirself untill a condition (that we call base state) is valid.

let's suppose we want to create a function that calculates the factorial of numbers we know that:

- 0! is equal to 1
- 1! is equal to 1 = 1\*0!

- $2!$  is equal to  $2 * 1 = 2 * 1!$
- $3!$  is equal to  $3 * 2 * 1 = 3 * 2!$
- $4!$  is equal to  $4 * 3 * 2 * 1 = 4 * 3!$
- $5!$  is equal to  $5 * 4 * 3 * 2 * 1 = 5 * 4!$  with that in mind, we can set the base condition as if  $n === 0$  we return 1, else we return  $n$  multiplied by the factorial of  $n-1$  and so on

```
function factorial(n){
  if (n === 0){
    return 1;
  }else{
    return n * factorial(n - 1);
  }
}

let n = 5;
let r = factorial(5); // r = 120
```

## Callbacks

A callback function is a function passed as an argument to another function, where it gets executed to complete a specific operation or task. This pattern allows for asynchronous control flow and is commonly used in event handling, array methods like `forEach()`.

```
function greet(name) {
  console.log(`Hello, ${name}!`);
}

function processUserInput(callback) {
  const name = "Alice"; // In a real app, this might come from user input
  callback(name);
}

processUserInput(greet);
```

In this example, `greet` is a callback function that's passed to `processUserInput`. The `processUserInput` function then calls the callback, passing it the name value.

## Callback hell

When we use callbacks to handle nested asynchronous operations, we can easily fall into "callback hell", a situation where our code becomes deeply nested and difficult to read. This

happens when each callback depends on the result of the previous one, forcing us to nest them inside each other.

```
getUser(userId, function(user) {  
  getPosts(user.id, function(posts) {  
    getComments(posts[0].id, function(comments) {  
      getReplies(comments[0].id, function(replies) {  
        console.log(replies); // Deeply nested!  
      });  
    });  
  });  
});
```

We can avoid callback hell by using either:

1. Promises ( `.then()` chains)
2. Async/await (synchronous-style syntax)

## Functional Programming Concepts

Functional programming breaks problems into smaller sub-problems, each solved by pure functions.

Functional programming cover the following 5 concepts

### Pure functions

These functions respect the following :

- They always produce the same output for same arguments irrespective of anything else.
- They have no side-effects i.e. they do not modify any arguments or local/global variables or input/output.
- They have immutability. The pure function's only result is the value it returns. They are deterministic.

### Recursive function

Iteration in functional languages is implemented through recursion. Recursive functions repeatedly call themselves until they reach the base case.

### First-Class functions and Higher-Order function

First-class functions are dealt with as first-class variable. The first class variables can be passed to functions as parameter, can be returned from functions or stored in data structures. Higher order functions are the functions that take other functions as arguments and they can also return functions.

## Referential transparency

In functional programs variables, once defined don't change their value throughout the program. Functional programs do not have assignment statements. If we have to store some value, we define new variables instead. This eliminates any chances of side effects because any variable can be replaced with its actual value at any point of execution. State of any variable is constant at any instant.

## Variables are Immutable

In functional programming, we can't modify a variable after it's been initialized. We can create new variables but we can't modify existing variables, and this really helps to maintain state throughout the runtime of a program. Once we create a variable and set its value, we can have full confidence knowing that the value of that variable will never change.

# Modules in Nodejs

## Introduction

In Node.js, modules are used to organize and reuse code by grouping related functions, classes, variables, and constants. Modules help maintain clean, modular, and scalable codebases and prevent naming conflicts through encapsulation. Node.js follows the CommonJS module system, where each file is treated as a separate module. We can export functionality from one file and import it into another.

## Built-in Core Modules

Node.js includes a set of core modules that provide essential functionalities like file system operations, HTTP servers, path manipulation, and more.

## Using Built-in Modules

We can use built-in modules by importing them with the `require` function:

```
const math = require('mathjs');

console.log(math.sqrt(25));
```

## Common Built-in Modules

| Module       | Description                                   |
|--------------|-----------------------------------------------|
| fs           | File system operations (read/write files)     |
| http / https | Create HTTP servers and clients               |
| path         | Handle file/directory paths                   |
| os           | Operating system-related utilities            |
| events       | Event-driven programming                      |
| crypto       | Cryptographic functions (hashing, encryption) |
| util         | Utility functions (e.g., promisify)           |

## Importing Functions

We can use **destructuring** to import only specific functionalities from a module instead of importing the entire module.

This is done using curly braces with the `require` statement, like so:

```
const { func1, func2 } = require('module-name');
```

## Creating Custom Modules

To create a custom Node.js module, we define a `.js` file (e.g., `myLibrary.js`) and export our functions, constants, or classes using `module.exports`. We can then import this module anywhere in our project using `require('./myLibrary')`.

## Example

`myLibrary.js`

```
const greet = name => `Hello, ${name}!`;
module.exports = { greet };
```

`main.js`

```
const { greet } = require('./myLibrary');
console.log(greet("Alice"));
```

# Splitting a Module into Multiple Files

When our library grows, we can split it into multiple files and combine them into one central module file.

We start by creating multiple files to store our functionality, then create an `index.js` file that requires all the modules. When we need to import them elsewhere, we simply import from `index.js`.

`index.js`

`index.js`

```
const greetings = require('./greetings');
const calculations = require('./calculations');
const libraryName = 'MyLibrary';
module.exports = { ...greetings, ...calculations, libraryName };
```

`greetings.js`

```
const greet = name => `Welcome to MyLibrary, ${name}!`;
module.exports = { greet };
```

`calculations.js`

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
module.exports = { add, subtract };
```

Now we can use:

```
const myLib = require('./myLibrary');
console.log(myLib.greet("Ali"));
console.log(myLib.add(5, 3));
```

## Packages

### Introduction

In Node.js, packages are reusable collections of code (Modules) that help us organize functionality, solve specific problems, and share solutions efficiently.

They often include libraries, configuration files, scripts, and metadata, and are managed

through npm (Node Package Manager).

Packages allow us to:

- Structure large projects modularly.
- Reuse and share code across apps.
- Distribute solutions across teams or publish them publicly through the npm registry.

## Built-in Packages

Node.js provides a variety of core packages that come pre-installed. These give us access to essential features like file handling, networking, path utilities, streams, buffers, and more.

We can import them directly using the `require()` function no need for installation.

**Example: Using `fs` and `path` modules:**

```
const fs = require('fs');
const path = require('path');

const filePath = path.join(__dirname, 'example.txt');
fs.writeFileSync(filePath, 'Hello, Node.js!');
console.log('File created successfully.');
```

## Installing Third-Party Packages

Node.js uses a centralized package registry called [npm](#). Here, we can discover and install thousands of packages contributed by developers around the world.

To install and use a third-party package, first we initialize our project using `npm init -y`, this creates a `package.json` file that keeps track of our dependencies.

```
my_project/
├─ node_modules/
├─ package.json
├─ index.js
└─ ...
```

After this, we install the package we want using `npm install package_name`. Then, we can import the package and use it in our script using the `require` function.

## Creating Custom Packages

Sometimes, we need to create our own packages either for internal use across multiple projects or to share with the community.

To create a custom package, we first create and initialize a new folder for it.

```
mkdir my-package
cd my-package
npm init -y
```

Then we create modules that our package should have

**Example** `my-package.js`

```
function greet(name) {
  return `Hello, ${name}!`;
}

module.exports = { greet };
```

After that, we need to set up the `package.json` file and add the configuration and information about our package, such as the name, version, author, and license.

```
{
  "name": "my-package",
  "version": "1.0.0",
  "main": "./my-package.js",
  "description": "A simple greeting package",
  "keywords": ["greet", "hello"],
  "author": "Our Name",
  "license": "MIT"
}
```

We can then add our package to a local project by including it in the `dependencies` section of the project's `package.json` file, and specifying the path to our package.

```
{
  "dependencies": {
    "my-package": "file:../my-package"
  }
}
```

```
}  
}
```

Then, to use it in our scripts, we simply import it using `require` .

```
const { greet } = require('my-package');  
console.log(greet('World'));
```

## Publishing Our Package

We can publish our package for others to use by creating account on [npmjs.com](https://npmjs.com) then we log in to our account using terminal by using `npm login` then to publish we just use `npm publish`

## Tasks

### Task 1

Create a function that tests whether a given number is a prime number or not.

### Task 2

Write a function that converts a decimal number to its binary representation using recursion.

### Task 3

Write a function `map(fn, arr)` that:

- Takes a callback function `fn` and an array `arr` .
- Returns a new array with `fn` applied to every element.

Test it with arrow function that doubles the values.